WEEK 13

Permissions, Notifications, WorkManager & Animations

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

This week your apps start working with the Android **system**: asking for **permissions**, showing **notifications**, and running reliable **background work** with WorkManager, even when the app is closed. You'll also make your UI feel polished with Compose **animations**. The project adds **reminders** to a to-do app.

Day	Topic	You will be able to...
Day 1	Runtime permissions	Request permissions correctly and handle "Deny"
Day 2	Notifications	Create channels and show notifications that open your app
Day 3	WorkManager basics	Run guaranteed background work with constraints
Day 4	Periodic and delayed work	Schedule repeating and delayed tasks
Day 5	Animations in Compose	Animate values, visibility, content and lists
Day 6	Weekly project	Add reminders to a to-do app

Tip: Animations were compile-checked with the real Compose compiler and libraries. Permissions, notifications and WorkManager are Android system APIs that couldn't be downloaded here, so that code was compile-checked against stand-ins with the same names and call shapes. Always test these features on a real phone or emulator (Android 13 or newer for the notification permission).

WEEK 13 • DAY 1

Runtime Permissions

Today's goal: Understand Android permissions and request them at runtime in Compose, including what to do when the user says no.

1. Why permissions?

Android protects private data and features (camera, location, contacts, notifications). An app must **declare** what it needs in the manifest, and for **dangerous** permissions it must also **ask the user** while the app is running.

Type	Examples	What you do
Normal	INTERNET, VIBRATE	Declare in the manifest. Granted automatically
Dangerous (runtime)	CAMERA, ACCESS_FINE_LOCATION, POST_NOTIFICATIONS (Android 13+), READ_CONTACTS	Declare in the manifest **and** request at runtime
Special	SCHEDULE_EXACT_ALARM, overlay	The user enables it in system Settings

AndroidManifest.xml

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

2. The request flow

Sketch: What happens when you ask

```
Already granted? --yes--> use the feature
      | no
      v
Show your own explanation first (why you need it)   <- good practice
      |
      v
System dialog:  [Allow]  [Don't allow]
      |                      |
  use feature          show a friendly message; if denied twice, Android won't show
                       the dialog again -> offer a button to open the app's Settings
```

3. Requesting a permission in Compose

Camera permission with rememberLauncherForActivityResult

```
@Composable
fun CameraPermissionScreen(onPermissionGranted: () -> Unit) {
    val context = LocalContext.current
    var deniedOnce by rememberSaveable { mutableStateOf(false) }

    val launcher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.RequestPermission()
    ) { granted ->
        if (granted) onPermissionGranted() else deniedOnce = true
    }

    fun isGranted() = ContextCompat.checkSelfPermission(context, Manifest.permission.CAMERA) ==
        PackageManager.PERMISSION_GRANTED

    Column(Modifier.fillMaxSize().padding(24.dp), verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally) {
        Icon(Icons.Default.Face, contentDescription = null, modifier = Modifier.size(64.dp))
        Spacer(Modifier.height(16.dp))
        Text("Scan documents with your camera", style = MaterialTheme.typography.titleLarge)
        Text("We only use the camera when you tap Scan. Photos stay on your phone.",
            textAlign = TextAlign.Center)
        Spacer(Modifier.height(24.dp))
        Button(onClick = {
            if (isGranted()) onPermissionGranted() else launcher.launch(Manifest.permission.CAMERA)
        }) { Text("Allow camera") }

        if (deniedOnce) {
            Spacer(Modifier.height(16.dp))
            Text("Camera permission is needed to scan. You can allow it in Settings.",
                color = MaterialTheme.colorScheme.error, textAlign = TextAlign.Center)
            TextButton(onClick = {
                val intent = Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS,
                    Uri.fromParts("package", context.packageName, null))
                context.startActivity(intent)
            }) { Text("Open Settings") }
        }
    }
}
```

Code	Meaning
rememberLauncherForActivityResult(RequestPermission())	Creates a launcher that shows the system dialog and gives you the result (true/false)
ContextCompat.checkSelfPermission(...)	Is it already granted? Check before asking
launcher.launch(Manifest.permission.CAMERA)	Shows the system permission dialog
ACTION_APPLICATION_DETAILS_SETTINGS	Opens your app's page in Settings, where the user can allow the permission manually

4. Best practices

Do	Don't
Ask **when the feature is used** (tap Scan → ask for the camera)	Ask for everything when the app first opens
Explain **why** before asking	Show the system dialog with no context
Let the app keep working without the permission when possible	Close the app or block everything if denied
Ask only for what you need (coarse location may be enough)	Request permissions "just in case"

> **Tip:** Many teams use Google's **Accompanist Permissions** library or their own helper for this. Check what your project already uses before writing a new one.

Practice Exercises

1. Write a **LocationPermissionScreen** that asks for ACCESS_COARSE_LOCATION with an explanation, and shows "Location allowed" when granted.

2. Add the "Open Settings" fallback to it, after the first denial.

3. In your team's app, find the manifest and list every permission. Which are dangerous (runtime) permissions?

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is the difference between normal and dangerous permissions?

Q2. Which function creates a permission launcher in Compose?

Q3. When is the best time to ask for a permission?

Q4. What happens after the user denies a permission twice?

Q5. Which notification permission is needed on Android 13+?

WEEK 13 • DAY 2

Notifications

Today's goal: Create a notification channel, ask for the notification permission, and show notifications that open the app.

1. Parts of a notification

Sketch: Anatomy

```
+-------------------------------------------------+
| (icon) My Tasks  -  now                         |   <- small icon, app name, time
| Call Amma                                       |   <- content title
| It's 7 PM. Don't forget to call Amma today!     |   <- content text (expandable with BigTextStyle)
|   [ Mark done ]   [ Snooze ]                    |   <- optional action buttons
+-------------------------------------------------+
```

2. Channels (Android 8.0 and newer)

Every notification belongs to a **channel**, for example "Reminders" or "Promotions". Users can turn each channel on or off in Settings. You must create the channel **before** posting, usually when the app starts.

3. A notification helper

Notifications.kt

```
object Notifications {
    const val REMINDERS_CHANNEL_ID = "reminders"

    fun createChannels(context: Context) {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel = NotificationChannel(
                REMINDERS_CHANNEL_ID,
                "Reminders",                                  // shown in the phone's Settings
                NotificationManager.IMPORTANCE_DEFAULT
            ).apply { description = "Reminders for your tasks" }
            context.getSystemService(NotificationManager::class.java).createNotificationChannel(channel)
        }
    }

    fun canPost(context: Context): Boolean =
        Build.VERSION.SDK_INT < Build.VERSION_CODES.TIRAMISU ||
            ContextCompat.checkSelfPermission(context, Manifest.permission.POST_NOTIFICATIONS) ==
            PackageManager.PERMISSION_GRANTED

    fun showReminder(context: Context, id: Int, title: String, text: String, openApp: PendingIntent) {
        if (!canPost(context)) return
        val notification = NotificationCompat.Builder(context, REMINDERS_CHANNEL_ID)
            .setSmallIcon(R.drawable.ic_notification)
            .setContentTitle(title)
            .setContentText(text)
            .setStyle(NotificationCompat.BigTextStyle().bigText(text))
            .setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .setContentIntent(openApp)                        // what happens on tap
            .setAutoCancel(true)                              // remove it after tapping
            .build()
        NotificationManagerCompat.from(context).notify(id, notification)
    }
}
```

Code	Meaning
if (SDK_INT >= O)	Channels exist only on Android 8.0 (API 26) and newer. Your minSdk 24 phones skip this
IMPORTANCE_DEFAULT / HIGH / LOW	How intrusive: sound, pop-up (heads-up) or silent
canPost(...)	On Android 13 (TIRAMISU) and newer, POST_NOTIFICATIONS must be granted
setSmallIcon(R.drawable.ic_notification)	A small, white-on-transparent icon (create it with New → Vector Asset)
setContentIntent(openApp)	Opens your app when tapped
notify(id, notification)	Shows it. The same id replaces an existing notification

4. Opening the app when tapped

A PendingIntent for MainActivity (Android only)

```
fun openAppIntent(context: Context): PendingIntent {
    val intent = Intent(context, MainActivity::class.java).apply {
        flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
    }
    return PendingIntent.getActivity(
        context, 0, intent,
        PendingIntent.FLAG_IMMUTABLE or PendingIntent.FLAG_UPDATE_CURRENT
    )
}
```

A **PendingIntent** is a "permission slip" that lets the system open your app later on your behalf. **FLAG_IMMUTABLE** is required on modern Android.

5. Create channels at app start, ask for permission at the right time

Application class (Android only)

```
@HiltAndroidApp
class TasksApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        Notifications.createChannels(this)     // safe to call every time; does nothing if it exists
    }
}
```

Asking for POST_NOTIFICATIONS when the user turns reminders on

```
@Composable
fun ReminderToggle(enabled: Boolean, onEnabledChange: (Boolean) -> Unit) {
    val context = LocalContext.current
    val launcher = rememberLauncherForActivityResult(ActivityResultContracts.RequestPermission()) { granted ->
        onEnabledChange(granted)
    }

    Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.padding(16.dp)) {
        Column(Modifier.weight(1f)) {
            Text("Daily reminders")
            Text("Get a notification at 7 PM", style = MaterialTheme.typography.bodySmall)
        }
        Switch(
            checked = enabled,
            onCheckedChange = { wantOn ->
                if (wantOn && Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU &&
                    ContextCompat.checkSelfPermission(context, Manifest.permission.POST_NOTIFICATIONS) !=
                    PackageManager.PERMISSION_GRANTED
                ) {
                    launcher.launch(Manifest.permission.POST_NOTIFICATIONS)
                } else {
                    onEnabledChange(wantOn)
                }
            }
        )
    }
}
```

Tip: Asking for the notification permission when the user turns on a feature that needs it (like this switch) gets far more "Allow" answers than asking on the first launch.

Practice Exercises

1. Add a second channel, "Promotions" (IMPORTANCE_LOW), to Notifications.createChannels.

2. Add a "Test notification" button to a screen that calls Notifications.showReminder. Try it with the permission allowed and denied.

3. Post two notifications with the same id. What happens? Then try different ids.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why do notifications need a channel?

Q2. When must you check POST_NOTIFICATIONS?

Q3. What does setContentIntent do?

Q4. What happens if you call notify with an id that's already showing?

Q5. Where should channels be created?

WEEK 13 • DAY 3

WorkManager Basics

Today's goal: Run guaranteed background work with WorkManager, add constraints, pass data, and observe the result.

1. Why WorkManager?

Tool	Survives leaving the screen?	Survives closing the app / reboot?	Use for
viewModelScope	No	No	Work for the current screen
WorkManager	Yes	**Yes**	Work that **must** finish: uploading, syncing, backups, reminders
AlarmManager	Yes	Yes (rescheduled after reboot)	Exact-time alarms (alarm clock apps)

Dependency (Android only)

```
implementation("androidx.work:work-runtime-ktx:2.9.1")
```

2. A worker

UploadWorker

```
class UploadWorker(
    appContext: Context,
    params: WorkerParameters
) : CoroutineWorker(appContext, params) {

    override suspend fun doWork(): Result {
        val noteId = inputData.getInt(KEY_NOTE_ID, -1)
        if (noteId == -1) return Result.failure()

        return try {
            delay(2000)                          // pretend: upload the note to the server
            Result.success(workDataOf(KEY_UPLOADED_AT to System.currentTimeMillis()))
        } catch (e: IOException) {
            if (runAttemptCount < 3) Result.retry() else Result.failure()
        }
    }

    companion object {
        const val KEY_NOTE_ID = "note_id"
        const val KEY_UPLOADED_AT = "uploaded_at"
    }
}
```

Code	Meaning
CoroutineWorker	A worker whose doWork() is a suspend function (runs on a background thread)
inputData.getInt(...)	Reads the data passed when the work was scheduled
Result.success(outputData)	Finished. Optionally returns data
Result.retry()	Try again later (WorkManager waits longer each time: "backoff")

Code	Meaning
Result.failure()	Give up
runAttemptCount	How many times this work has been tried

3. Scheduling the work

Enqueue a one-time request with constraints

```
class UploadWorker(appContext: Context, params: WorkerParameters) : CoroutineWorker(appContext, params) {
    override suspend fun doWork(): Result = Result.success()
}

fun scheduleUpload(context: Context, noteId: Int): java.util.UUID {
    val constraints = Constraints.Builder()
        .setRequiredNetworkType(NetworkType.CONNECTED)       // only with internet
        .setRequiresBatteryNotLow(true)
        .build()

    val request = OneTimeWorkRequestBuilder<UploadWorker>()
        .setInputData(workDataOf("note_id" to noteId))
        .setConstraints(constraints)
        .addTag("uploads")
        .build()

    WorkManager.getInstance(context).enqueueUniqueWork(
        "upload_note_$noteId",
        ExistingWorkPolicy.REPLACE,                          // replace an older upload of the same note
        request
    )
    return request.id
}
```

If there's no internet, the work **waits**. When the phone reconnects (even hours later, even after a restart), WorkManager runs it. That's the guarantee.

4. Observing the work's status

Show upload progress in the UI

```
class UploadViewModel(private val workManager: WorkManager) : ViewModel() {
    fun statusOf(id: java.util.UUID): StateFlow<String> =
        workManager.getWorkInfoByIdFlow(id)
            .map { info ->
                when (info?.state) {
                    WorkInfo.State.ENQUEUED -> "Waiting for internet..."
                    WorkInfo.State.RUNNING -> "Uploading..."
                    WorkInfo.State.SUCCEEDED -> "Uploaded"
                    WorkInfo.State.FAILED -> "Upload failed"
                    else -> ""
                }
            }
            .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), "")
}
```

Tip: To inject repositories into a worker with Hilt, use **@HiltWorker** and **@AssistedInject constructor(@Assisted context, @Assisted params, repo)**, plus the androidx.hilt:hilt-work library. Your team's project may already have this set up.

5. Testing it

- Turn off Wi-Fi and mobile data, schedule the upload, and check that the status says "Waiting for internet...".
- Turn the internet back on: the work runs within a short time.
- Use **App Inspection** → **Background Task Inspector** in Android Studio to see all scheduled work and its state.

Practice Exercises

1. Write a **CleanupWorker** that deletes notes older than 30 days (call a fake function) and returns success.

2. Schedule it with the constraint setRequiresCharging(true).

3. Make UploadWorker return failure if the note id is missing, and show "Upload failed" in the UI.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. When should you use WorkManager instead of viewModelScope?

Q2. What does Result.retry() do?

Q3. What do constraints do?

Q4. How do you pass data to a worker?

Q5. What does enqueueUniqueWork with REPLACE do?

WEEK 13 • DAY 4

Periodic and Delayed Work

Today's goal: Schedule repeating and delayed work, cancel it, and know when an exact alarm is needed instead.

1. Periodic work

A daily sync

```
class SyncWorker(appContext: Context, params: WorkerParameters) : CoroutineWorker(appContext, params) {
    override suspend fun doWork(): Result {
        delay(1000)                 // pretend: download the latest data and save it to Room
        return Result.success()
    }
}

fun scheduleDailySync(context: Context) {
    val request = PeriodicWorkRequestBuilder<SyncWorker>(1, TimeUnit.DAYS)
        .setConstraints(
            Constraints.Builder().setRequiredNetworkType(NetworkType.UNMETERED).build()   // Wi-Fi only
        )
        .build()

    WorkManager.getInstance(context).enqueueUniquePeriodicWork(
        "daily_sync",
        ExistingPeriodicWorkPolicy.KEEP,       // if it's already scheduled, keep the existing one
        request
    )
}
```

Rule	Detail
Minimum interval	**15 minutes**. Smaller values are raised to 15 minutes
Timing is approximate	Android batches background work to save battery. A "daily" job runs about once a day, not at an exact time
Always use unique work	Otherwise every app start schedules another copy
KEEP vs UPDATE	KEEP leaves an existing schedule alone; UPDATE applies your new settings to it

2. Delayed one-time work: a reminder

Remind me in 2 hours

```
class ReminderWorker(appContext: Context, params: WorkerParameters) : CoroutineWorker(appContext, params) {
    override suspend fun doWork(): Result {
        val title = inputData.getString("title") ?: return Result.failure()
        // Day 2: Notifications.showReminder(applicationContext, id, "Reminder", title, openApp)
        return Result.success()
    }
}

fun scheduleReminder(context: Context, taskId: Int, title: String, delayMinutes: Long) {
    val request = OneTimeWorkRequestBuilder<ReminderWorker>()
        .setInitialDelay(delayMinutes, TimeUnit.MINUTES)
        .setInputData(workDataOf("title" to title, "task_id" to taskId))
        .build()

    WorkManager.getInstance(context)
        .enqueueUniqueWork("reminder_$taskId", ExistingWorkPolicy.REPLACE, request)
}

fun cancelReminder(context: Context, taskId: Int) {
    WorkManager.getInstance(context).cancelUniqueWork("reminder_$taskId")
}
```

Using the task's id in the unique name (**reminder_7**) lets you **replace** the reminder when the user changes the time, and **cancel** it when the task is completed or deleted.

3. Calculating the delay to a time of day

Minutes until 7 PM (plain Kotlin, run)

```
import java.time.LocalDateTime
import java.time.LocalTime
import java.time.Duration

fun minutesUntil(target: LocalTime, now: LocalDateTime): Long {
    var next = now.toLocalDate().atTime(target)
    if (!next.isAfter(now)) next = next.plusDays(1)          // already passed today -> tomorrow
    return Duration.between(now, next).toMinutes()
}

fun main() {
    val sevenPm = LocalTime.of(19, 0)
    println(minutesUntil(sevenPm, LocalDateTime.of(2026, 10, 16, 18, 30)))   // 30 minutes
    println(minutesUntil(sevenPm, LocalDateTime.of(2026, 10, 16, 9, 0)))     // 10 hours
    println(minutesUntil(sevenPm, LocalDateTime.of(2026, 10, 16, 20, 15)))   // tomorrow
}
```

Output

```
30
600
1365
```

Pass the result to setInitialDelay. In the app, use **LocalDateTime.now()** for now. This logic is plain Kotlin, so it's easy to test.

4. Exact times: when WorkManager isn't enough

Need	Use
"Sometime today", sync, backup, approximate reminders	WorkManager
An alarm clock or medicine reminder at **exactly** 7:00	AlarmManager.setExactAndAllowWhileIdle (needs the SCHEDULE_EXACT_ALARM / USE_EXACT_ALARM permission rules)
Work that the user is watching right now (music, navigation)	A foreground service

Careful: Phones in battery-saving mode may delay WorkManager tasks. For most apps that's fine and good for battery. Only use exact alarms when timing really matters, because Google Play reviews their use.

Practice Exercises

1. Schedule a periodic "backup" worker every 6 hours that only runs on Wi-Fi and while charging.

2. Write **rescheduleReminder(taskId, newTitle, newDelay)** using REPLACE, and test that only one reminder exists afterwards (Background Task Inspector).

3. Extend minutesUntil with a test for exactly 19:00 (it should return 1440, the same time tomorrow). Run it.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is the minimum interval for periodic work?

Q2. Why use enqueueUniquePeriodicWork?

Q3. How do you run a one-time task after a delay?

Q4. How do you cancel one task's reminder?

Q5. When do you need AlarmManager instead?

WEEK 13 • DAY 5

Animations in Compose

Today's goal: Animate values, appearance, content changes and list items to make the UI feel smooth.

1. Animating a value: animate*AsState

Size and colour that animate

```
@Composable
fun SelectableCard(selected: Boolean, onClick: () -> Unit) {
    val scale by animateFloatAsState(
        targetValue = if (selected) 1.05f else 1f,
        animationSpec = spring(dampingRatio = Spring.DampingRatioMediumBouncy),
        label = "scale"
    )
    val background by animateColorAsState(
        targetValue = if (selected) MaterialTheme.colorScheme.primaryContainer
                      else MaterialTheme.colorScheme.surfaceVariant,
        label = "background"
    )
    val corner by animateDpAsState(targetValue = if (selected) 24.dp else 8.dp, label = "corner")

    Box(
        modifier = Modifier
            .padding(16.dp)
            .graphicsLayer { scaleX = scale; scaleY = scale }
            .clip(RoundedCornerShape(corner))
            .background(background)
            .clickable(onClick = onClick)
            .padding(24.dp)
    ) {
        Text(if (selected) "Selected!" else "Tap me")
    }
}
```

You only describe the **target** value. Whenever the target changes, Compose animates smoothly from the current value to the new one. **graphicsLayer** scales the drawing without changing the layout, which is efficient.

animationSpec	Feels like
tween(durationMillis = 300)	A fixed-length, smooth animation
spring(dampingRatio = ...)	A natural, physical movement (can bounce)
keyframes { }	Full control over values at specific times

2. Showing and hiding: AnimatedVisibility

Custom enter and exit animations

```
@Composable
fun ErrorBanner(message: String?) {
    AnimatedVisibility(
        visible = message != null,
        enter = slideInVertically { fullHeight -> -fullHeight } + fadeIn(),
        exit = slideOutVertically { fullHeight -> -fullHeight } + fadeOut()
    ) {
        Surface(color = MaterialTheme.colorScheme.errorContainer, modifier = Modifier.fillMaxWidth()) {
            Text(message ?: "", modifier = Modifier.padding(16.dp))
        }
    }
}
```

3. Switching content: AnimatedContent and Crossfade

An animated counter and a crossfade between states

```
@Composable
fun AnimatedCounter(count: Int) {
    AnimatedContent(
        targetState = count,
        transitionSpec = {
            if (targetState > initialState) {
                (slideInVertically { it } + fadeIn()) togetherWith (slideOutVertically { -it } + fadeOut())
            } else {
                (slideInVertically { -it } + fadeIn()) togetherWith (slideOutVertically { it } + fadeOut())
            }
        },
        label = "counter"
    ) { value ->
        Text("$value", style = MaterialTheme.typography.displayMedium)
    }
}

enum class LoadState { LOADING, CONTENT }

@Composable
fun LoadingCrossfade(state: LoadState) {
    Crossfade(targetState = state, label = "load") { s ->
        when (s) {
            LoadState.LOADING -> CircularProgressIndicator()
            LoadState.CONTENT -> Text("Here is your data")
        }
    }
}
```

4. Repeating animations

A pulsing recording dot

```
@Composable
fun RecordingDot() {
    val transition = rememberInfiniteTransition(label = "pulse")
    val alpha by transition.animateFloat(
        initialValue = 1f,
        targetValue = 0.2f,
        animationSpec = infiniteRepeatable(tween(800), RepeatMode.Reverse),
        label = "alpha"
    )
    Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.padding(16.dp)) {
        Box(Modifier.size(12.dp).alpha(alpha).clip(CircleShape).background(MaterialTheme.colorScheme.error))
        Spacer(Modifier.width(8.dp))
        Text("Recording")
    }
}
```

5. Animating size and list items

animateContentSize and animateItem

```
@Composable
fun ExpandingText(text: String) {
    var expanded by remember { mutableStateOf(false) }
    Text(
        text = text,
        maxLines = if (expanded) Int.MAX_VALUE else 2,
        overflow = TextOverflow.Ellipsis,
        modifier = Modifier
            .animateContentSize()                 // the height changes smoothly
            .clickable { expanded = !expanded }
            .padding(16.dp)
    )
}

@Composable
fun AnimatedTaskList(tasks: List<String>, onRemove: (String) -> Unit) {
    LazyColumn {
        items(tasks, key = { it }) { task ->
            Row(
                verticalAlignment = Alignment.CenterVertically,
                modifier = Modifier.animateItem().fillMaxWidth().padding(horizontal = 16.dp)
            ) {
                Text(task, modifier = Modifier.weight(1f))
                IconButton(onClick = { onRemove(task) }) {
                    Icon(Icons.Default.Close, contentDescription = "Remove $task")
                }
            }
        }
    }
}
```

Modifier.animateItem() animates items when they're added, removed or reordered in a lazy list. It needs stable **keys** (Week 5), another reason to always use them.

> **Tip:** Good animation is **quick and subtle** (150 to 400 ms). It should help users understand what changed, not slow them down. Respect the system's "Remove animations" accessibility setting where your team requires it.

Practice Exercises

1. Make a favourite IconButton whose heart grows (scale 1.3) and changes colour with a spring when tapped.

2. Use AnimatedVisibility to show a "Back to top" button only when the list has been scrolled.

3. Animate a progress bar: animateFloatAsState from 0f to the target progress when a value changes.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What do you give animateFloatAsState?

Q2. Which API animates showing and hiding content?

Q3. What is the difference between AnimatedContent and Crossfade?

Q4. What does Modifier.animateItem() need to work well?

Q5. How long should most UI animations be?

WEEK 13 • DAY 6

Weekly Project: Task Reminders

Today's goal: Build a to-do list where each task can get a reminder notification, scheduled with WorkManager.

1. How it fits together

Sketch: Flow of a reminder

```
User taps (bell) on "Call Amma" -> chooses "In 1 hour"
     -> TasksViewModel.setReminder() -> ReminderScheduler.schedule()
     -> WorkManager: OneTimeWork "reminder_7", delay 60 min       (survives closing the app)
     ... 1 hour later ...
     -> TaskReminderWorker.doWork() -> shows a notification "Task reminder: Call Amma"
Completing or deleting the task -> scheduler.cancel(7) -> cancelUniqueWork("reminder_7")
```

2. Scheduler and worker

TaskReminders.kt

```
data class Task(val id: Int, val title: String, val isDone: Boolean = false, val remindAtMinutes: Long? = null)

interface ReminderScheduler {
    fun schedule(task: Task, delayMinutes: Long)
    fun cancel(taskId: Int)
}

class WorkManagerReminderScheduler(private val context: Context) : ReminderScheduler {
    override fun schedule(task: Task, delayMinutes: Long) {
        val request = OneTimeWorkRequestBuilder<TaskReminderWorker>()
            .setInitialDelay(delayMinutes, TimeUnit.MINUTES)
            .setInputData(workDataOf("task_id" to task.id, "title" to task.title))
            .build()
        WorkManager.getInstance(context).enqueueUniqueWork("reminder_${task.id}", ExistingWorkPolicy.REPLACE, request)
    }

    override fun cancel(taskId: Int) {
        WorkManager.getInstance(context).cancelUniqueWork("reminder_$taskId")
    }
}

class TaskReminderWorker(appContext: Context, params: WorkerParameters) : CoroutineWorker(appContext, params) {
    override suspend fun doWork(): Result {
        val taskId = inputData.getInt("task_id", -1)
        val title = inputData.getString("title") ?: return Result.failure()
        val canPost = Build.VERSION.SDK_INT < Build.VERSION_CODES.TIRAMISU ||
            ContextCompat.checkSelfPermission(applicationContext, Manifest.permission.POST_NOTIFICATIONS) ==
            PackageManager.PERMISSION_GRANTED
        if (canPost) {
            val notification = NotificationCompat.Builder(applicationContext, "reminders")
                .setSmallIcon(R.drawable.ic_notification)
                .setContentTitle("Task reminder")
                .setContentText(title)
                .setAutoCancel(true)
                .build()
            NotificationManagerCompat.from(applicationContext).notify(taskId, notification)
        }
        return Result.success()
    }
}
```

The ViewModel depends on the **ReminderScheduler interface**, not on WorkManager directly. In Week 14 you'll test the ViewModel with a fake scheduler, with no Android needed.

3. ViewModel

TasksViewModel.kt

```
data class TasksUiState(val tasks: List<Task> = emptyList(), val reminderFor: Task? = null)

class TasksViewModel(private val scheduler: ReminderScheduler) : ViewModel() {
    private var nextId = 1
    private val _uiState = MutableStateFlow(TasksUiState())
    val uiState: StateFlow<TasksUiState> = _uiState.asStateFlow()

    fun addTask(title: String) {
        if (title.isBlank()) return
        _uiState.update { it.copy(tasks = it.tasks + Task(nextId++, title.trim())) }
    }

    fun toggleDone(task: Task) {
        val updated = task.copy(isDone = !task.isDone)
        if (updated.isDone) scheduler.cancel(task.id)            // no reminder for finished tasks
        _uiState.update { s -> s.copy(tasks = s.tasks.map { if (it.id == task.id) updated else it }) }
    }

    fun onReminderClick(task: Task) = _uiState.update { it.copy(reminderFor = task) }
    fun onReminderDismiss() = _uiState.update { it.copy(reminderFor = null) }

    fun setReminder(task: Task, minutes: Long) {
        scheduler.schedule(task, minutes)
        _uiState.update { s ->
            s.copy(
                tasks = s.tasks.map { if (it.id == task.id) it.copy(remindAtMinutes = minutes) else it },
                reminderFor = null
            )
        }
    }

    fun delete(task: Task) {
        scheduler.cancel(task.id)
        _uiState.update { s -> s.copy(tasks = s.tasks - task) }
    }
}
```

4. Screen

TasksScreen.kt (with animations)

```
@Composable
fun TasksScreen(
    uiState: TasksUiState,
    onAdd: (String) -> Unit,
    onToggle: (Task) -> Unit,
    onReminderClick: (Task) -> Unit,
    onSetReminder: (Task, Long) -> Unit,
    onReminderDismiss: () -> Unit,
    onDelete: (Task) -> Unit
) {
    var draft by rememberSaveable { mutableStateOf("") }

    Column(Modifier.fillMaxSize().padding(16.dp)) {
        Row(verticalAlignment = Alignment.CenterVertically) {
            OutlinedTextField(draft, { draft = it }, label = { Text("New task") },
                singleLine = true, modifier = Modifier.weight(1f))
            IconButton(onClick = { onAdd(draft); draft = "" }) { Icon(Icons.Default.Add, contentDescription = "Add task") }
        }
        LazyColumn {
            items(uiState.tasks, key = { it.id }) { task ->
                Row(
                    verticalAlignment = Alignment.CenterVertically,
                    modifier = Modifier.animateItem().fillMaxWidth()
                ) {
                    Checkbox(checked = task.isDone, onCheckedChange = { onToggle(task) })
```

```
                    Column(Modifier.weight(1f)) {
                        Text(task.title, textDecoration = if (task.isDone) TextDecoration.LineThrough else null)
                        AnimatedVisibility(visible = task.remindAtMinutes != null && !task.isDone) {
                            Text("Reminder in ${task.remindAtMinutes} min", style = MaterialTheme.typography.bodySmall,
                                color = MaterialTheme.colorScheme.primary)
                        }
                    }
                    IconButton(onClick = { onReminderClick(task) }, enabled = !task.isDone) {
                        Icon(Icons.Default.Notifications, contentDescription = "Set reminder")
                    }
                    IconButton(onClick = { onDelete(task) }) { Icon(Icons.Default.Delete, contentDescription = "Delete") }
                }
            }
        }
    }

    uiState.reminderFor?.let { task ->
        AlertDialog(
            onDismissRequest = onReminderDismiss,
            title = { Text("Remind me about \"${task.title}\"") },
            text = {
                Column {
                    listOf(15L to "In 15 minutes", 60L to "In 1 hour", 180L to "In 3 hours").forEach { (minutes, label) ->
                        TextButton(onClick = { onSetReminder(task, minutes) }) { Text(label) }
                    }
                }
            },
            confirmButton = { },
            dismissButton = { TextButton(onClick = onReminderDismiss) { Text("Cancel") } }
        )
    }
}
```

5. Wiring it up

Creating the ViewModel and permission (Android only)

```
// Application.onCreate(): Notifications.createChannels(this)   (channel id "reminders")
@Composable
fun TasksRoute() {
    val context = LocalContext.current
    val viewModel: TasksViewModel = viewModel { TasksViewModel(WorkManagerReminderScheduler(context.applicationContext)) }
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()
    // Ask for POST_NOTIFICATIONS the first time the user taps a bell (see Day 2's ReminderToggle)
    TasksScreen(uiState, viewModel::addTask, viewModel::toggleDone, viewModel::onReminderClick,
        viewModel::setReminder, viewModel::onReminderDismiss, viewModel::delete)
}
// With Hilt: @HiltViewModel + @Inject constructor(scheduler: ReminderScheduler) and a @Binds.
```

6. Test it

- Add a task, set a reminder for 15 minutes, then **close the app completely**. The notification still arrives.
- For quicker testing, temporarily add a "1 minute" option.
- Set a reminder, then tick the task as done: the reminder must not appear (check the Background Task Inspector).
- Deny the notification permission and set a reminder: the worker runs but doesn't show anything, and the app doesn't crash.

7. Project challenges

- Let the user pick an exact time with a TimePicker and use minutesUntil (Day 4) to calculate the delay.
- Add a "Mark done" action button to the notification.
- Save tasks in Room (Week 11) so they survive restarts, and inject everything with Hilt (Week 12).
- Add a daily summary notification at 8 PM using periodic work.

Week 13 checklist

I can...	Done?
Explain normal, dangerous and special permissions	[ ]
Request a runtime permission in Compose and handle denial	[ ]
Create a notification channel and show notifications	[ ]
Handle POST_NOTIFICATIONS on Android 13+	[ ]
Write a CoroutineWorker and schedule it with constraints	[ ]
Schedule periodic, delayed and unique work, and cancel it	[ ]
Choose between WorkManager, AlarmManager and viewModelScope	[ ]
Animate values, visibility, content and list items	[ ]
Build the task reminders project	[ ]

Quick Quiz

Q1. Why does the reminder still arrive after closing the app?

Q2. Why is each reminder unique work named reminder_?

Q3. Why does the ViewModel use a ReminderScheduler interface?

Q4. What happens in the worker if the notification permission was denied?

Q5. Which modifier animates rows being added and removed?

Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

Day 1: Runtime Permissions

Exercise 1

```
@Composable
fun LocationPermissionScreen() {
    val context = LocalContext.current
    var granted by remember {
        mutableStateOf(ContextCompat.checkSelfPermission(context, Manifest.permission.ACCESS_COARSE_LOCATION) ==
            PackageManager.PERMISSION_GRANTED)
    }
    val launcher = rememberLauncherForActivityResult(ActivityResultContracts.RequestPermission()) { granted = it }

    Column(Modifier.padding(24.dp)) {
        if (granted) {
            Text("Location allowed")
        } else {
            Text("We use your approximate location to show the weather near you.")
            Button(onClick = { launcher.launch(Manifest.permission.ACCESS_COARSE_LOCATION) }) { Text("Allow location") }
        }
    }
}
```

Exercise 2

```
@Composable
fun OpenSettingsButton() {
    val context = LocalContext.current
    TextButton(onClick = {
        context.startActivity(Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS,
            Uri.fromParts("package", context.packageName, null)))
    }) { Text("Open Settings") }
}
```

Quiz answers

Q1. Normal ones are granted automatically from the manifest; dangerous ones must also be requested at runtime.

Q2. rememberLauncherForActivityResult with ActivityResultContracts.RequestPermission()

Q3. When the user uses the feature that needs it, after explaining why.

Q4. Android stops showing the dialog; the app should offer to open Settings.

Q5. POST_NOTIFICATIONS

Day 2: Notifications

Exercise 1

```
fun createPromotionsChannel(context: Context) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val channel = NotificationChannel("promotions", "Promotions", NotificationManager.IMPORTANCE_LOW)
        context.getSystemService(NotificationManager::class.java).createNotificationChannel(channel)
    }
}
```

Exercise 2

```
@Composable
fun TestNotificationButton(onTest: () -> Unit) {
    // onTest = { Notifications.showReminder(context, 1, "Test", "Hello from my app!", openAppIntent(context)) }
    Button(onClick = onTest) { Text("Test notification") }
}
```

Exercise 3

```
// Same id: the second notification replaces the first (only one is visible).
// Different ids: both notifications are shown.
fun notifyTwice(context: Context, first: android.app.Notification, second: android.app.Notification) {
    NotificationManagerCompat.from(context).notify(1, first)
    NotificationManagerCompat.from(context).notify(1, second)
}
```

Quiz answers

Q1. Since Android 8.0, every notification must belong to a channel, which users can control in Settings.

Q2. On Android 13 (API 33) and newer.

Q3. Sets the PendingIntent that runs (e.g. opens the app) when the notification is tapped.

Q4. The existing notification is replaced.

Q5. When the app starts, for example in Application.onCreate().

Day 3: WorkManager Basics

Exercise 1

```
class CleanupWorker(appContext: Context, params: WorkerParameters) : CoroutineWorker(appContext, params) {
    override suspend fun doWork(): Result {
        deleteNotesOlderThan(days = 30)
        return Result.success()
    }
    private suspend fun deleteNotesOlderThan(days: Int) { delay(100) }
}
```

Exercise 2

```
class CleanupWorker(appContext: Context, params: WorkerParameters) : CoroutineWorker(appContext, params) {
    override suspend fun doWork(): Result = Result.success()
}

fun scheduleCleanup(context: Context) {
    val request = OneTimeWorkRequestBuilder<CleanupWorker>()
        .setConstraints(Constraints.Builder().setRequiresCharging(true).build())
        .build()
    WorkManager.getInstance(context).enqueue(request)
}
```

Exercise 3

```
class UploadWorker(appContext: Context, params: WorkerParameters) : CoroutineWorker(appContext, params) {
    override suspend fun doWork(): Result {
        val noteId = inputData.getInt("note_id", -1)
        if (noteId == -1) return Result.failure()       // the UI maps FAILED to "Upload failed"
        return Result.success()
    }
}
```

Quiz answers

Q1. When the work must finish even if the user leaves the screen or closes the app.

Q2. Asks WorkManager to run the work again later, with increasing delays.

Q3. Make the work wait until conditions are met, such as having an internet connection.

Q4. With setInputData(workDataOf(...)), read in the worker with inputData.

Q5. Cancels any existing work with the same name and schedules the new one.

Day 4: Periodic and Delayed Work

Exercise 1

```
class BackupWorker(appContext: Context, params: WorkerParameters) : CoroutineWorker(appContext, params) {
    override suspend fun doWork(): Result = Result.success()
}

fun scheduleBackup(context: Context) {
    val request = PeriodicWorkRequestBuilder<BackupWorker>(6, TimeUnit.HOURS)
        .setConstraints(Constraints.Builder()
            .setRequiredNetworkType(NetworkType.UNMETERED)
            .setRequiresCharging(true)
            .build())
        .build()
    WorkManager.getInstance(context).enqueueUniquePeriodicWork("backup", ExistingPeriodicWorkPolicy.KEEP, request)
}
```

Exercise 2

```
class ReminderWorker(appContext: Context, params: WorkerParameters) : CoroutineWorker(appContext, params) {
    override suspend fun doWork(): Result = Result.success()
}

fun rescheduleReminder(context: Context, taskId: Int, newTitle: String, newDelayMinutes: Long) {
    val request = OneTimeWorkRequestBuilder<ReminderWorker>()
        .setInitialDelay(newDelayMinutes, TimeUnit.MINUTES)
        .setInputData(workDataOf("title" to newTitle))
        .build()
    WorkManager.getInstance(context).enqueueUniqueWork("reminder_$taskId", ExistingWorkPolicy.REPLACE, request)
}
```

Exercise 3

```
import java.time.Duration
import java.time.LocalDateTime
import java.time.LocalTime

fun minutesUntil(target: LocalTime, now: LocalDateTime): Long {
    var next = now.toLocalDate().atTime(target)
    if (!next.isAfter(now)) next = next.plusDays(1)
    return Duration.between(now, next).toMinutes()
}

fun main() {
    println(minutesUntil(LocalTime.of(19, 0), LocalDateTime.of(2026, 10, 16, 19, 0)))
}
```

Output

```
1440
```

Quiz answers

Q1. 15 minutes.

Q2. So the job isn't scheduled again on every app start.

Q3. OneTimeWorkRequestBuilder with setInitialDelay(...).

Q4. Give each reminder a unique name (like reminder_7) and call cancelUniqueWork with it.

Q5. When something must happen at an exact time, like an alarm clock.

Day 5: Animations in Compose

Exercise 1

```
@Composable
fun FavouriteButton() {
    var liked by remember { mutableStateOf(false) }
    val scale by animateFloatAsState(if (liked) 1.3f else 1f,
        animationSpec = spring(dampingRatio = Spring.DampingRatioMediumBouncy), label = "scale")
    val tint by animateColorAsState(
        if (liked) MaterialTheme.colorScheme.error else MaterialTheme.colorScheme.onSurface, label = "tint")
    IconButton(onClick = { liked = !liked }) {
        Icon(if (liked) Icons.Filled.Favorite else Icons.Outlined.FavoriteBorder,
            contentDescription = if (liked) "Unlike" else "Like",
            tint = tint, modifier = Modifier.graphicsLayer { scaleX = scale; scaleY = scale })
    }
}
```

Exercise 2

```
@Composable
fun ListWithBackToTop(items: List<String>) {
    val listState = rememberLazyListState()
    val showButton by remember { derivedStateOf { listState.firstVisibleItemIndex > 3 } }
    val scope = rememberCoroutineScope()
    Box {
        LazyColumn(state = listState) { items(items, key = { it }) { Text(it, Modifier.padding(16.dp)) } }
        AnimatedVisibility(visible = showButton, modifier = Modifier.align(Alignment.BottomEnd).padding(16.dp)) {
            FloatingActionButton(onClick = { scope.launch { listState.animateScrollToItem(0) } }) {
                Icon(Icons.Default.KeyboardArrowUp, contentDescription = "Back to top")
            }
        }
    }
}
```

Exercise 3

```
@Composable
fun AnimatedProgress(target: Float) {
    val progress by animateFloatAsState(targetValue = target, animationSpec = tween(600), label = "progress")
    LinearProgressIndicator(progress = { progress }, modifier = Modifier.fillMaxWidth().padding(16.dp))
}
```

Quiz answers

Q1. The target value; Compose animates from the current value to it.

Q2. AnimatedVisibility

Q3. Crossfade only fades; AnimatedContent lets you define any enter/exit transition.

Q4. Stable keys on the lazy list items.

Q5. Short, about 150 to 400 ms.

Day 6: Weekly Project: Task Reminders

Quiz answers

Q1. WorkManager keeps the scheduled work and runs it even if the app was closed.

Q2. So it can be replaced when changed and cancelled when the task is done or deleted.

Q3. So it doesn't depend on WorkManager directly and can be tested with a fake.

Q4. It skips showing the notification and still returns success.

Q5. Modifier.animateItem()

Next: When you finish this week, tell your mentor and ask for the **Week 14** material. Great work, Anjana!